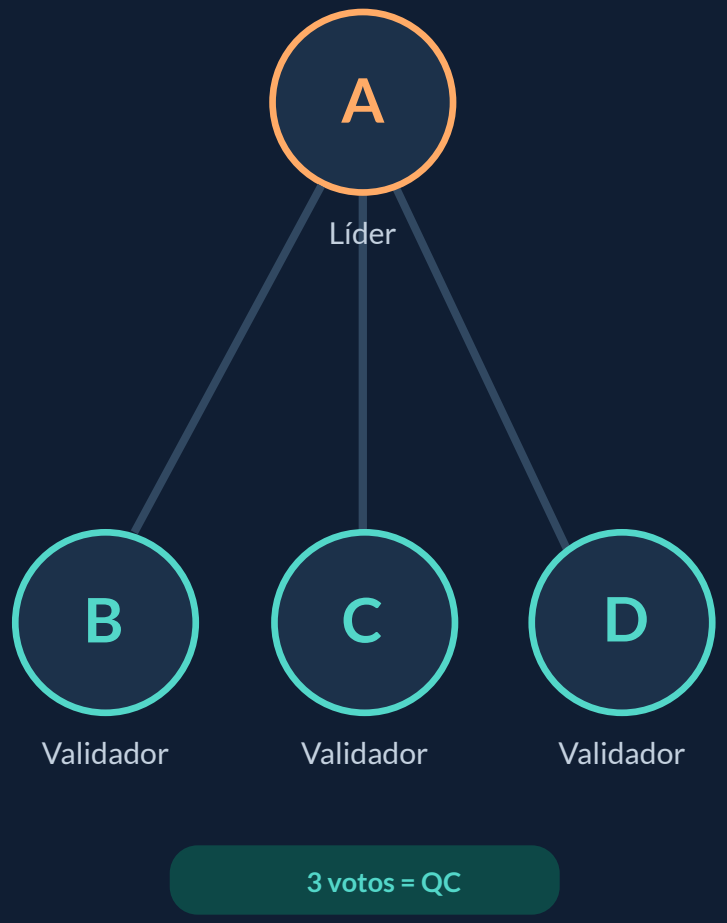
HOTSTUFF

Consenso BFT explicado desde cero

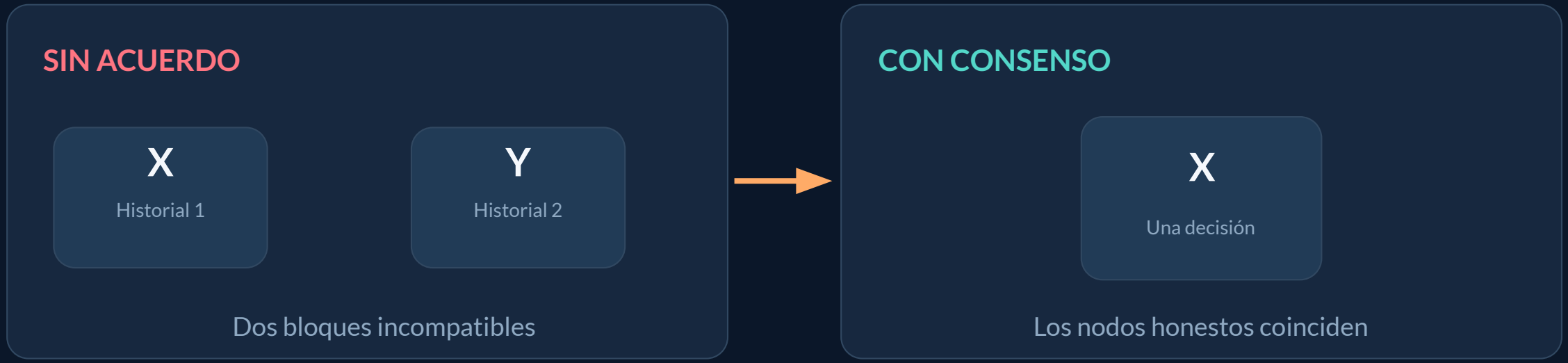
Una historia · Cuatro validadores · Tres votos

Iago Otero Goce
Alicia JiaJun Lorenzo Lourido



¿Por qué necesitamos consenso?

Si cada ordenador acepta una historia distinta, la blockchain deja de ser fiable.



Objetivo: acordar el orden de los bloques SIN tener que confiar en cada validador.

Validador y bloque NO son lo mismo

VALIDADOR = ORDENADOR

Comprueba propuestas y emite votos.



BLOQUE = DATOS

Agrupa transacciones que queremos registrar.

BLOQUE X

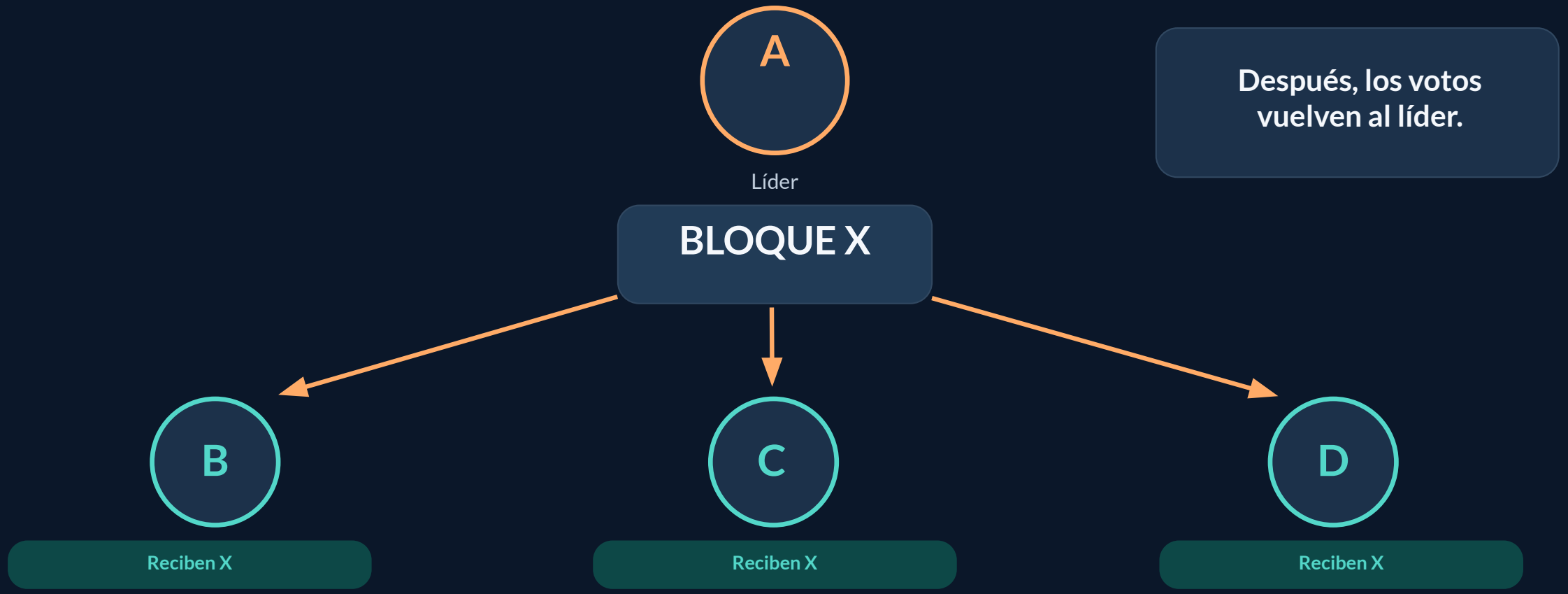
Transacción 1 · Ana → Luis

Transacción 2 · Pedro → María

Hash del bloque anterior …

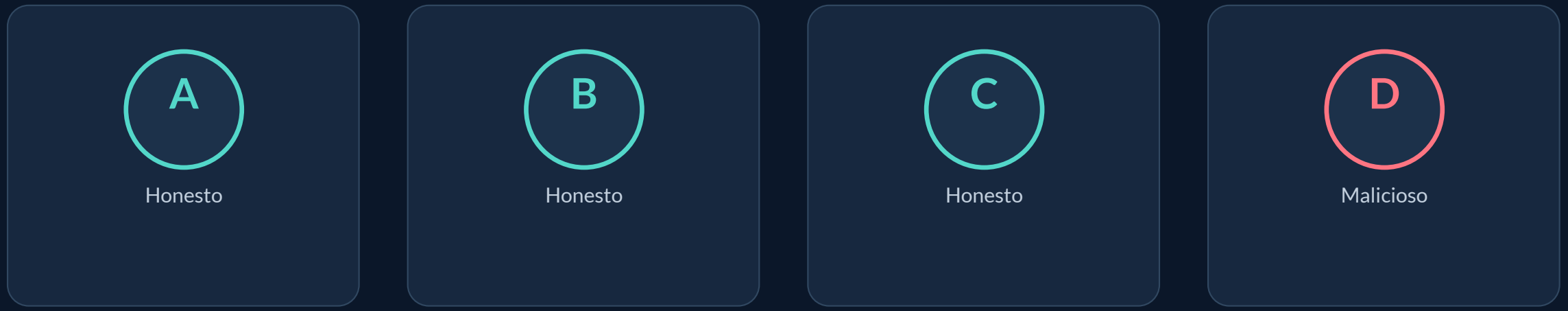
Un mismo bloque X puede estar copiado en los 4 ordenadores.

El líder propone X; todos comprueban X



¿Por qué con 4 nodos hacen falta 3 votos?

Ejemplo: 4 validadores, 1 puede ser bizantino.



$$n = 3f + 1$$

$$QC = 2f + 1 = 3 \text{ votos}$$

Con 3 votos válidos y compatibles se puede formar un QC. Un QC por sí solo no finaliza el bloque.

HotStuff: líder + votos + certificados QC

LÍDER

Propone un bloque y coordina la recogida de votos.

VOTOS

Cada validador honesto comprueba y responde.

QC

Prueba de que hubo suficientes votos válidos.

PACEMAKER + LOCK

Cambio de vista y reglas de voto seguro.

Diseñado para redes parcialmente síncronas

Basic HotStuff: SIEMPRE el mismo bloque X

Las fases se completan en orden para una propuesta.

1 PREPARE

Votos → prepareQC

2 PRE-COMMIT

Votos → precommitQC

3 COMMIT

Votos → commitQC

4 DECIDE

Decisión final

BLOQUE X · BLOQUE X · BLOQUE X · BLOQUE X

Los nodos votan simultáneamente DENTRO de cada fase; las fases son sucesivas.

En Basic, A, B, C y D votan en la misma fase

Fase	A	B	C	D
Prepare	Vota X	Vota X	Vota X	Vota X
Pre-Commit	Vota X	Vota X	Vota X	Vota X
Commit	Vota X	Vota X	Vota X	Vota X
Decide	Decisión	Decisión	Decisión	Decisión

Cada fila se completa antes de pasar a la siguiente.

Chained HotStuff: las fases se SOLAPAN entre bloques

Cada vista propone un bloque nuevo; sus votos también ayudan a confirmar antecesores.

	VISTA 1	VISTA 2	VISTA 3	VISTA 4
BLOQUE X	Prepare	Pre-Commit	Commit	Decide
BLOQUE Y	—	Prepare	Pre-Commit	Commit
BLOQUE Z	—	—	Prepare	Pre-Commit
BLOQUE W	—	—	—	Prepare

Es un esquema LÓGICO: una votación por nueva propuesta hace avanzar varias etapas a la vez.

Basic y Chained: la diferencia

BASIC

Un mismo bloque X

- Prepare
- Pre-Commit
- Commit
- Decide

Se termina X y luego se trabaja en Y.

CHAINED

Bloques X, Y, Z, W encadenados

- Vista 1: X
- Vista 2: Y → X
- Vista 3: Z → Y → X
- Vista 4: W → Z → Y → X

Se avanza en varios bloques al mismo tiempo.

Seguridad no significa “poder avanzar siempre”

SAFETY · SEGURIDAD

Dos nodos honestos no deben decidir de forma definitiva bloques incompatibles. Se protege mediante votos, QC y reglas de bloqueo.

LIVENESS · VIVACIDAD

La red debe poder seguir produciendo decisiones cuando hay suficiente participación y se cumplen las condiciones de comunicación.

Ejemplo: si 2 de 4 no responden, no hay QC; se para el progreso, no se “aprueba mal” por ello.

Ataques: ¿qué intenta hacer el adversario?

LÍDER MALICIOSO

Propone X a unos y Y a otros. Las reglas de voto y el cuórum evitan certificar ramas contradictorias dentro del límite BFT.

NODO BIZANTINO

Envía votos incorrectos, contradictorios o deja de colaborar. Las firmas y verificaciones detectan mensajes inválidos.

DoS / JAMMING

Satura o bloquea el tráfico de red. Puede impedir que lleguen los votos necesarios.

PARTICIÓN DE RED

Separa los nodos en dos grupos de 2. Ninguno reúne por sí solo 3 votos: se para el progreso.

No todo ataque rompe la seguridad: muchos atacan principalmente la disponibilidad.

HotStuff fuera del laboratorio

Tres referencias para diferenciar despliegues reales, evolución del protocolo y antecedentes históricos.

FLOW

Red blockchain en producción.

Consenso basado en HotStuff; evolución Jolteon.

APTOS

AptosBFT: evolución de DiemBFT y Jolteon.

Uso en una red de contratos inteligentes.

LIBRA / DIEM

Proyecto histórico de pagos.

LibraBFT/DiemBFT influyó en sistemas posteriores.

Importante: «usa HotStuff» no significa necesariamente «implementa exactamente Basic HotStuff».

De los casos reales a PharmaChain

Caso académico propuesto: trazabilidad de lotes farmacéuticos entre organizaciones.

CUATRO VALIDADORES

A · Fabricante

B · Distribuidor

C · Farmacia

D · Organismo regulador

Con n = 4, tolerancia de diseño f = 1.

EVENTO DEMOSTRABLE

Registrar el lote L-104 y su transferencia.

Tres votos compatibles forman QC.

Simular un líder malicioso o nodo desconectado.

Comprobar seguridad y progreso.

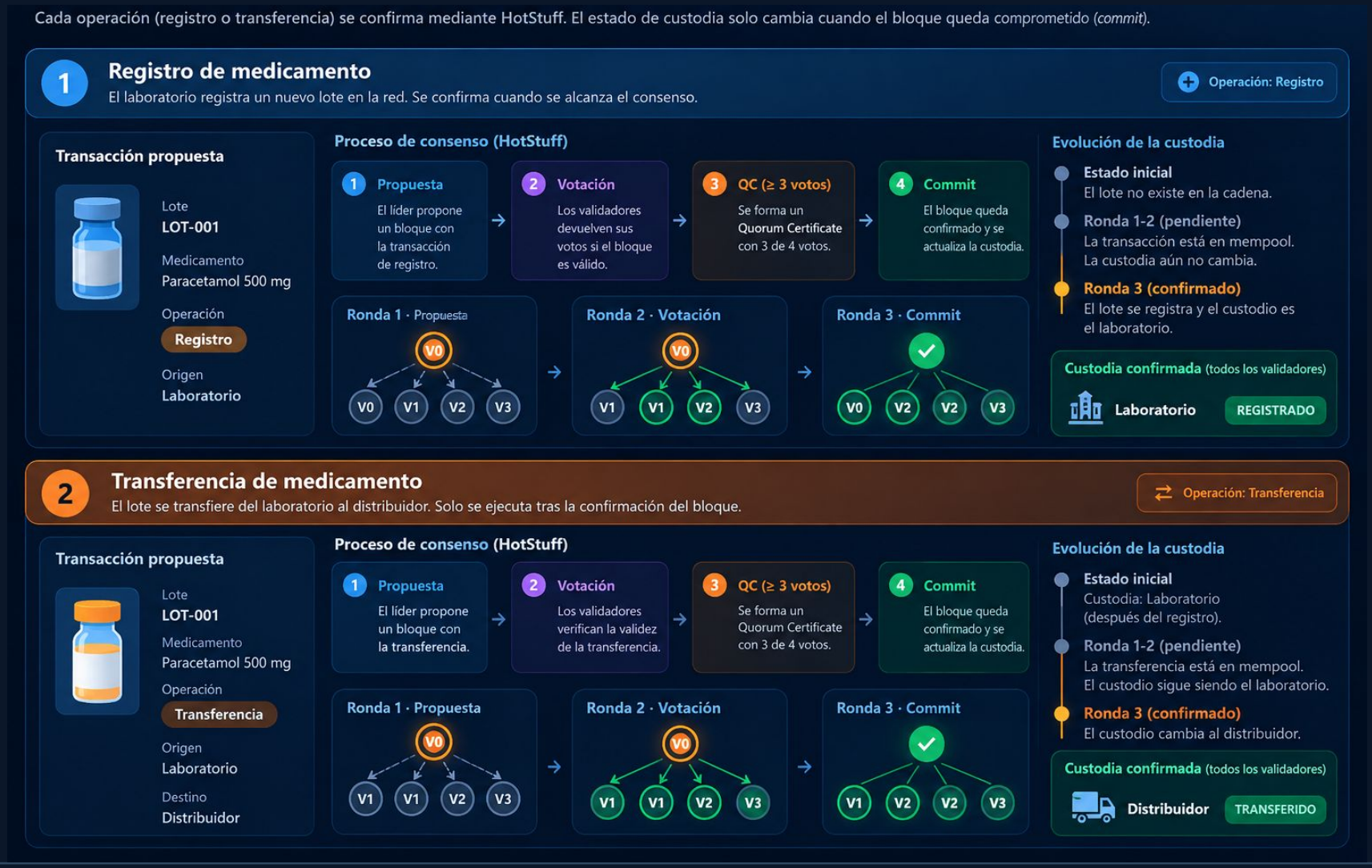
Límite: el consenso valida el acuerdo sobre registros, NO que los datos físicos sean verdaderos.

Demo : 4 nodos, 8 escenarios



Enlace del repositorio de la demo: https://github.com/ajlorenzo1315/HotStuff-PharmaChain

Medicamentos · consenso y custodia confirmada



Ventajas e inconvenientes

VENTAJAS

- ✓ Votos recogidos por un líder y QC compactos
- ✓ Comunicación lineal en el modelo original
- ✓ Finalidad determinista dentro de las hipótesis
- ✓ Sin minería PoW de alto coste energético

LIMITACIONES

- No elige por sí solo quién puede validar
- Sin cuórum, el protocolo deja de progresar
- Depende de la comunicación y del Pacemaker
- El líder y la red pueden ser cuellos de botella

Referencias y materiales

[1] Yin et al. (2019). HotStuff: BFT Consensus with Linearity and Responsiveness. arxiv.org/abs/1803.05069

[2] HotStuff original (implementación). github.com/hot-stuff/libhotstuff

[3] relab/hotstuff (framework de investigación). github.com/relab/hotstuff

[4] Flow (2023). Jolteon: Advancing Flow’s Consensus Algorithm. flow.com

[5] Flow, código de consenso. github.com/onflow/flow-go/tree/master/consensus/hotstuff

[6] Aptos. Aptos White Paper: Consensus. aptos.dev/network/blockchain/aptos-white-paper

[7] Aptos Core, código fuente. github.com/aptos-labs/aptos-core

[8] Diem, repositorio histórico (LibraBFT / DiemBFT). github.com/diem/diem

[9] EOS Network (2024). Spring 1.0 / Savanna Consensus. eosnetwork.com

[10] Material docente: «Protocolos de consenso», p. 47.